Team 45: LES Parts Database UI
Bi-Weekly Update 3

Anthony Beneventi, Jamaal Bloom, Jaswin Jabbal

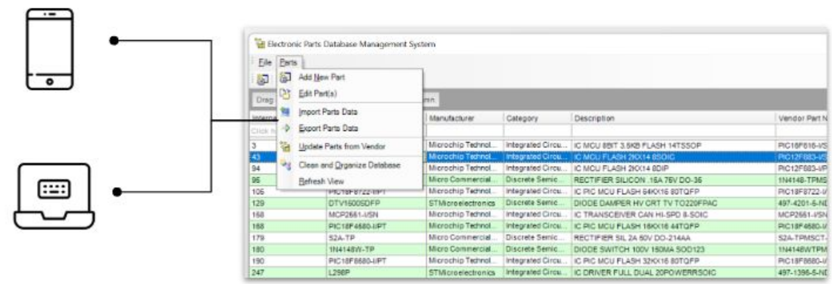
Sponsor: John Lusher

TA: Sabyasachi Gupta

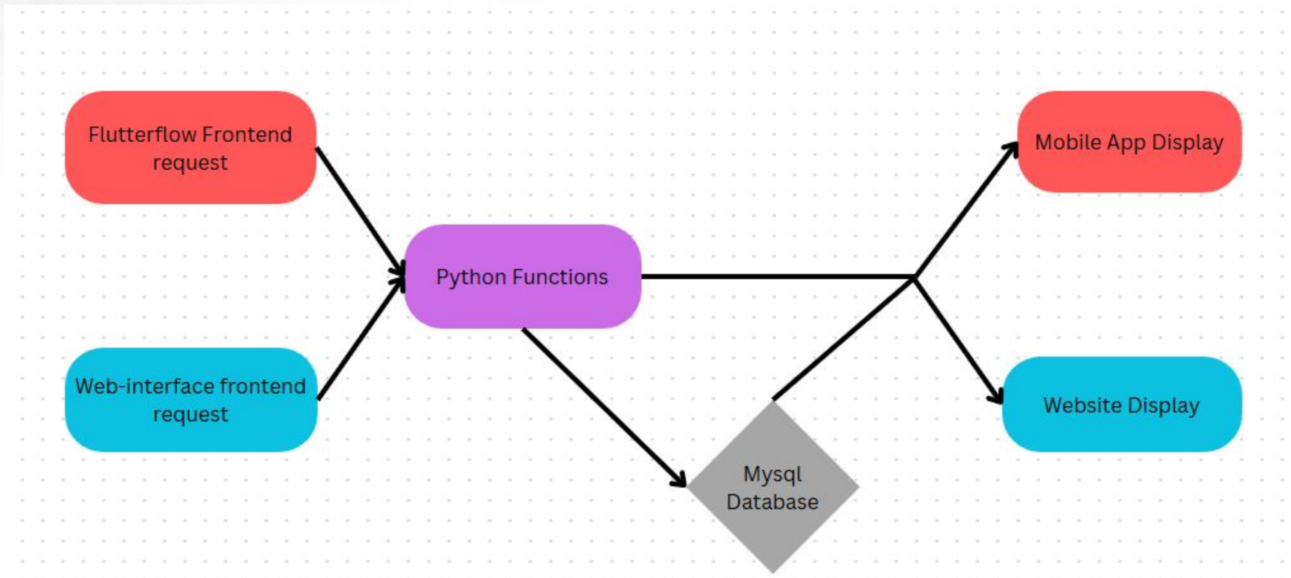
Project Summary

- The current database requires manually updating parts and limits multiple users working at the same time.

- Develop a web and mobile application that will allow for the auto population and filtering of part information



Project/Subsystem Overview



Team:
Jamaal:
Anthony:
Jaswin:

Project Timeline

Subsystem Designs and Testing (completed 1/27)	Start work on a singular database (completed 2/20)	Connect backend API to frontend (to complete by 2/27)	Final Integration (to complete by 3/6)	System Test (to complete by 3/17)	Validation (to complete by 3/31)	Demo and Report (to complete by 4/21)

Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

Web Application UI

Jaswin Jabbal

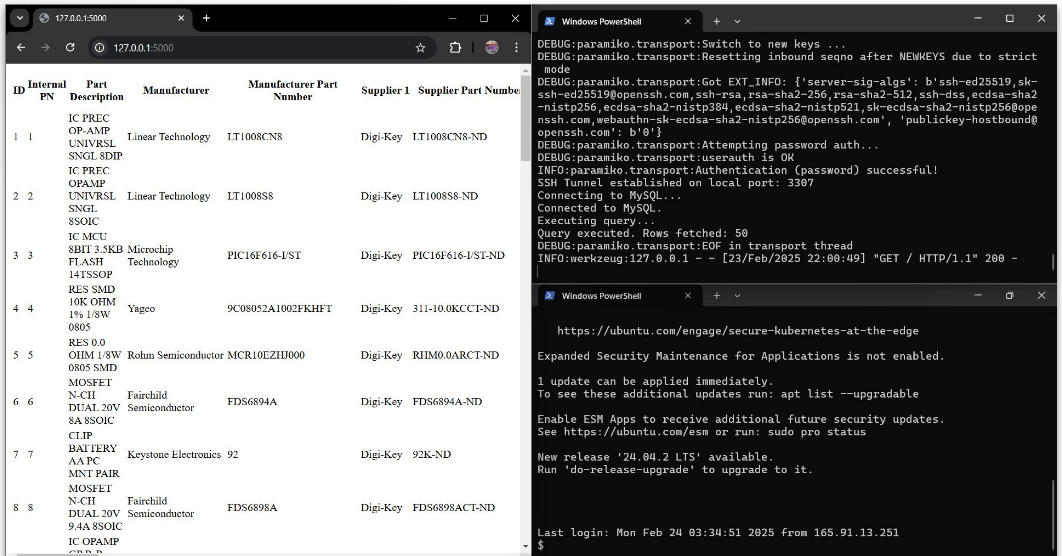
Accomplishments since Update 2 10+ hrs of effort	Ongoing progress/problems and plans until the next presentation
- <u>Debugging led me to build a test app from ground-up</u> **Tested a local Flask instance that connects securely to the remote MySQL database through an SSH tunnel - able to display parts to HTML!** - <u>Model Validation</u> **Integration underway with Jamaal's CRUD functions - need to validate with test routes and existing UI routes.**	- <u>Route Validation</u> **Now that Dr. Lusher's database is able to display data locally via python, the next part of my integration is to secure this MySQL connection to my existing UI app. For this:** - <u>Schedule meeting with Appseed Flask team</u> **To discuss relation between Flask-Migrate, SQLAlchemy and how to use mysql.connector with open-source configuration!** - Deployment Environment by 3/17

```python
# Function to connect to DB through SSH tunnel
def create_db_connection():
    tunnel = SSHTunnelForwarder(
        ssh_address_or_host=ssh_config['ssh_address_or_host'],
        ssh_username=ssh_config['ssh_username'],
        ssh_password=ssh_config['ssh_password'],
        remote_bind_address=ssh_config['remote_bind_address'],
        local_bind_address=('127.0.0.1', 3307)
    )
    tunnel.start()

    conn = mysql.connector.connect(
        host='127.0.0.1',
        port=3307,
        user=mysql_config['user'],
        password=mysql_config['password'],
        database=mysql_config['database']
    )

    return conn, tunnel
```

Currently testing local CRUD routes



Local Flask instance successfully displays parts - maintained by MySQL monitor and constant SSH tunnel

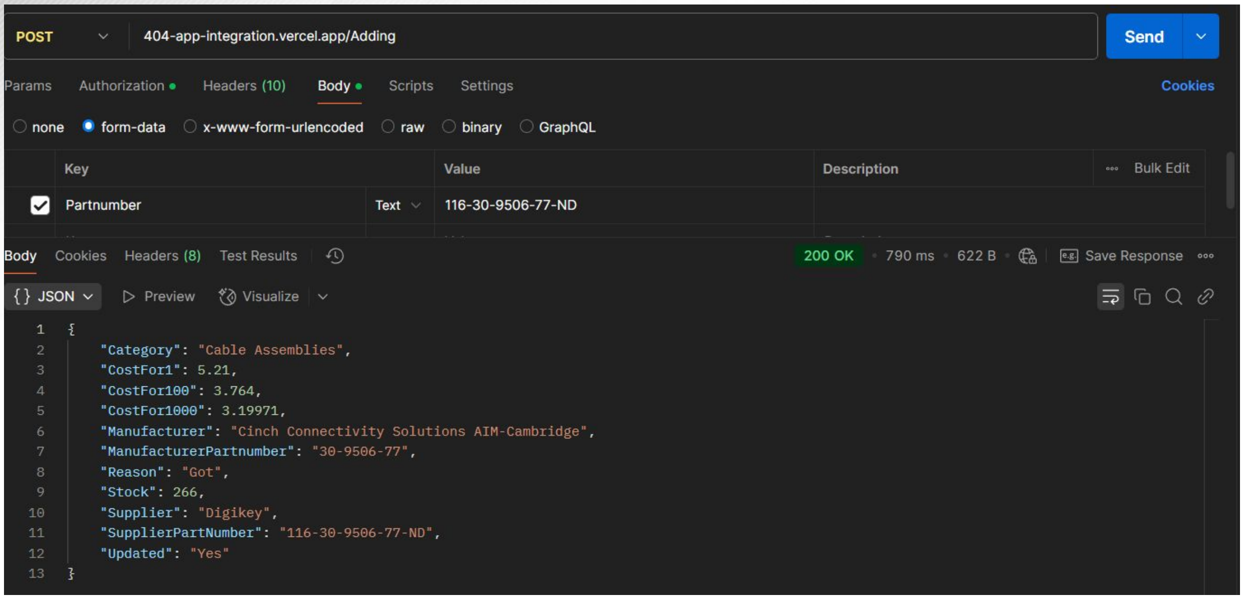
DB connection via remote & local bind

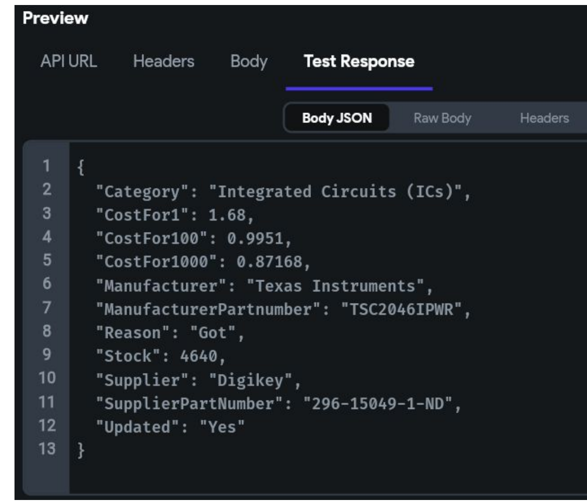
Python Backend

Jamaal Bloom

Accomplishments since Update 2 25 hrs of effort	Ongoing progress/problems and plans until the next presentation
• Local python connected with online database • Post request for Adding tested on Flutterflow and base api callers	• Fix the timeout issue. (Believe that SSH is the problem, Solution: find deployment websites until one of them work. • Format outputs

Python Backend





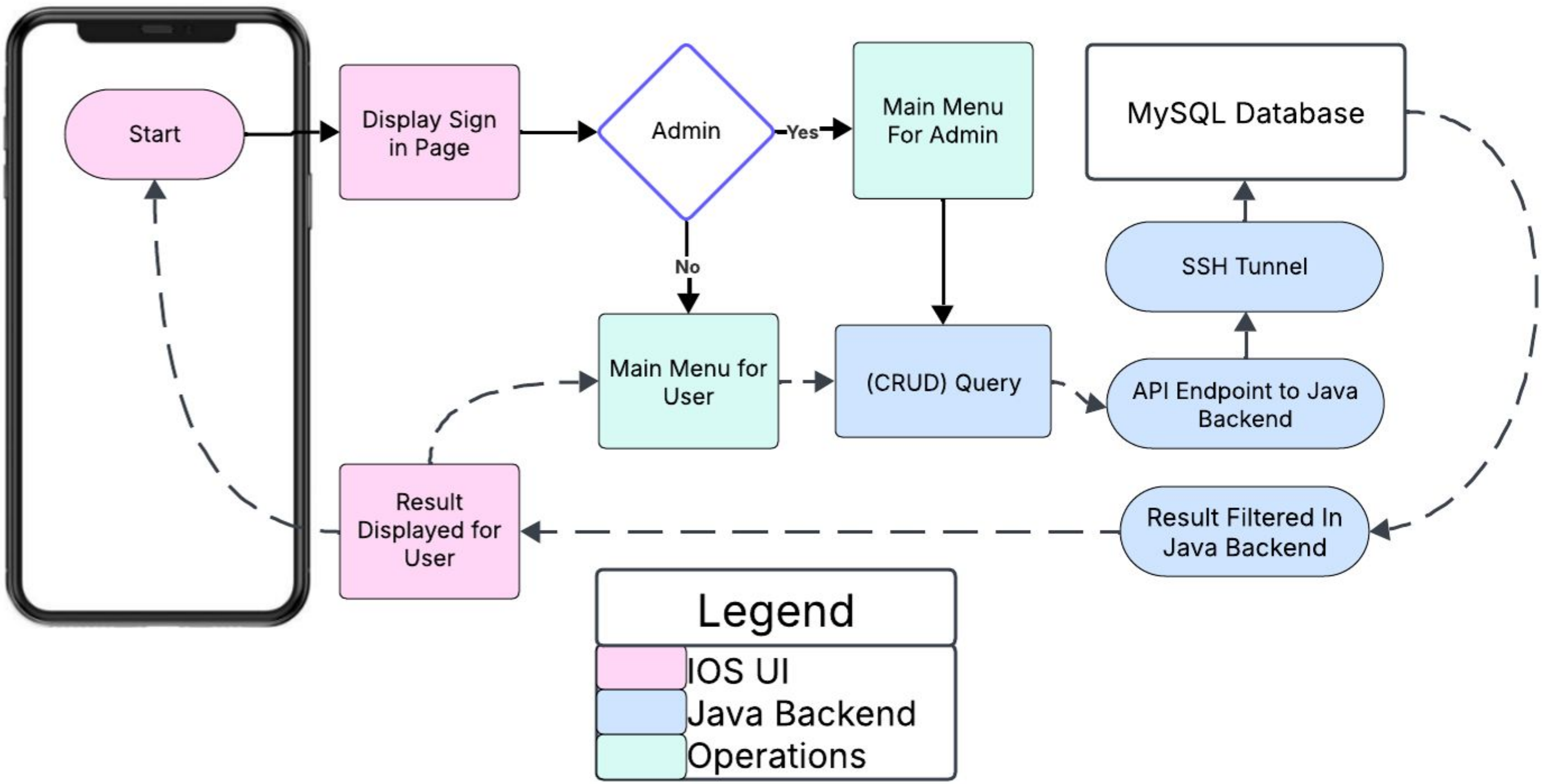
IOS Application

Anthony Beneventi

Accomplishments since Update 2 25+ hrs of effort	Ongoing progress/problems and plans until the next presentation
• SSH Tunnel Connection to online Database • Abandoned previous cloud platform for lack of SSH support, and implemented application into EC2.	• SSH Tunnel permissions error preventing full connection • Rework of connection type from cloud to ssh. • User Authority and making the App/UI look good

IOS Subsystem

Anthony Beneventi



Execution Plan

Task Assignments	Owner	1/13/25	1/20/25	1/27/25	2/3/25	2/10/25	2/17/25	2/24/25	3/3/25	3/10/25	3/17/25	3/24/25	3/31/25	4/7/25	4/14/25	4/21/25	4/28/25	Due Date
Finish Subsystems	Team																	
Status Update #1	Team																	1/27/25
Convert code to callable API	Jamaal																	
New models based on DB requirement	Jaswin																	
Fix Update/Insert	Anthony																	
Status Update #2	Team																	2/10/25
Connect with online database	Jamaal																	
Routes based on new models	Jaswin																	
Integrate Database	Anthony																	
Status Update #3	Team																	2/24/25
Format frontend and backend requests	Jamaal																	
Data Consistency	Jaswin																	
Consolidate App	Anthony																	
Finish general integration	Team																	
Status Update #4	Team																	3/17/25
Implement failsafes	Jamaal																	
Deployment	Jaswin																	
System validation	Anthony																	
Staus Update #5	Team																	3/31/25
Finalize code	Jamaal																	
Deployment	Jaswin																	
Optimize code	Anthony																	
Final Presentation	Team																	4/14/25
Showcase/ Demo	Team																	4/24/25

Key	
Planned	
In Progress	
Complete	
Behind Schedule	

Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

Validation Plan

Pararagph #	Test Name	Success Criteria	Methodology	Status	Responsible Engineer(s
3.2.1.4	Database Sync	The web and mobile applications uses the same database and changes visible from both	Attempt changes from multiple devices and ensure consistency	Untested	Jamaal
3.2.1.1	API Correctness	The API pulls correct information from the supplier	Compare recieved information with suppliers website	TESTED	Jamaal
3.2.6.2	Request Correctness	The backend correctly recieves and handles the frontend requests	Send frontend request from both applications and examine results	Untested	Jamaal
3.2.1.3	Multiple Requests	Application can handle multiple requests at the same time	Send multiple requests	Untested	Jamaal
3.2.2.1	Model Validation	All DB models align correctly with the updated schema used by Jamaal	Compare Flask SQLAlchemy models against the latest database schem	Untested	Jaswin
3.2.4.1	Route Functionality	Form submission routes to the correct display without errors	Manually fill out and submir forms in the application, verifying data correctness	Untested	Jaswin
3.2.4.2	Frontend Data Consistency	Submitted data is correctly displayed on a corresponding UI template	Manual validation by cross-checking displayed records with DB entries	Untested	Jaswin
3.2.6.1	Deployment Environment	All required dependencies are correctly installed and configured on the end machine	Validate with Docker, verify installation logs, ensure WGSI server	Untested	
3.2.1.2	CRUD Operation Test	Pulling apart information from the MySQL Database	Access a local MySQL Server with dummy datta	Untested	Anthony
3.2.1.4	Updated Database	The API Correctly updates the entire database	Checking previous and updated dababases to ensure completion	Untested	Anthony
3.2.1.5	Request Correctness	Having all requested components, such as User authority	Giving specific permission to individual User admins	Untested	Anthony
3.2.1.3	Application Deployed	Uploaded on the cloud and ready for use	Able to be sent for beta testing	Untested	Anthony

Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Team 45: Anthony Beneventi, Jamaal Bloom, and Jaswin Jabbal

Questions?